Testing Collaboration

As A Team Sport

Juggling priorities across Dev, Ops, and QA for predictable delivery.

Speaker

Haidee Fontanes

Process Improvement Manager/System Integration Lead

Date

April 29, 2026

StarEast Conference

Scenario 01

The Release Slip

A critical project is delayed by two weeks. The post-mortem begins, and the finger-pointing starts.

Development

"We finished coding on time. Testing Testing started way too late."

Velocity Focus

Quality Assurance

"The requirements changed three times during the sprint."

Coverage Focus

Operations

"The staging environment wasn't ready because no one booked it."

Stability Focus

The Consequence

Everyone is technically "right" — yet the business still loses.

Impact

-2 Weeks Delay

Critical Incident

Sprint Two: The Breakdown

CURRENT TIME
08:00 AM

Environment Down

OFFLINE

The staging environment crashed during the nightly build. Logs are inaccessible.

OWNER: OPS

ETA: UNKNOWN

Data Missing

INCOMPLETE

Test accounts for the new feature haven't been created. created. Payment gateway mocks are failing.

OWNER: DATA TEAM

BLOCKING: 3 TESTS

Devs Offshore

UNAVAILABLE

Key developers who can fix the blocker are in India (IST). Their shift ended 1 hour ago.

OWNER: DEV LEAD

DELAY: 14 HOURS

ESCALATION FROM IT DIRECTOR

DEADLINE: 12:00 PM

"This situation needs to be resolved by noon, and testing must be completed by the end of the day."

At this moment, the issue is no longer technical. **It is organizational.**

Quality is *not* owned by QA. Quality is owned by **the delivery team.**

> ⚠ *"Siloed excellence creates **systemic failure**."*

Core Insight • Testing Collaboration

Session Objectives

What You'll Take Away Today

Actionable insights tailored for every role in the delivery pipeline.

For QA Pros

Quality Assurance

Move from **gatekeeping** to integration leadership.

Gain earlier influence on requirements, test data, and environments.

Shift focus from "finding bugs" to "preventing risks".

For Developers

Engineering

Shorten feedback loops and drastically reduce rework.

Align defect prioritization directly with sprint goals.

Stop "throwing code over the wall" and start co-owning quality.

For Ops & Execs

Delivery Leadership

Build delivery predictability and reduce late-stage risk.

Make ownership and escalation paths explicit and clear.

Transform testing from a cost center to a delivery accelerator.

Goal: Transform testing from a bottleneck into a stabilizing force.

Section 2.1

Why Testing Breaks Under Pressure

→ Diagnosing the root causes: **misaligned ownership**, priorities, and timing.

Root Cause Analysis

The Three Silos: Local Optimization

Development

Optimizing for Velocity

Primary Goals

- Feature completion speed
- Sprint burn-down
- Technical innovation

The Risk

"Works on my machine" mentality; late integration causing massive rework.

Operations

Optimizing for Stability

Primary Goals

- Environment uptime
- Release readiness & security
- Minimizing production change

The Risk

Rigid gates that slow delivery; environments that drift from production reality.

Quality Assurance

Optimizing for Coverage

Primary Goals

- Test case execution
- Defect identification
- Requirement traceability

The Risk

Testing quality into the product too late; acting as a bottleneck gatekeeper.

Result: When each function optimizes locally, **the system as a whole fails.**

Systemic Breakdown

Root Causes Across the Timeline

How small misalignments compound into delivery failure.

Requirements Shift

Scope changes late without adjusting test plans. Testability ignored.

PLANNING

DEVELOPMENT

Sprint Gaps

"Dev complete" doesn't mean "Test ready." High WIP creates bottlenecks.

Env Not Ready

Data incomplete. Integration points failing. Testing blocked.

DELAY

CRISIS

Compressed Window

2 weeks becomes 2 days.
Coverage drops.
Risk skyrockets.

Costly Failure

Escalation, blame, and expensive late-stage fixes.

RELEASE

The Reality: Issues are discovered when change is most expensive—at the end.

Cost of Change

100x Higher

Section 2.2

Redefining Testing as a Shared Responsibility

→ From Gatekeeper to **Integrator**.

Paradigm Shift

Traditional vs. Collaborative Model

Traditional Model

Siloed & Reactive

QA as Gatekeeper

Testing acts as a final checkpoint, blocking releases and creating bottlenecks.

Testing at the End

Quality activities start only after development is "complete."

Defect Detection

Focus is on finding bugs that already exist in the code.

Functional Ownership

QA owns quality; if a bug escapes, it's "QA's fault."

Collaborative Model

Integrated & Proactive

QA as Integrator

Testing connects development and operations to ensure flow.

Testing Throughout

Quality activities embedded in requirements, sprints, and ops.

Risk Prevention

Focus is on preventing defects before code is even written.

Shared Accountability

The entire delivery team owns the quality outcome.

Executive Insight

Organizations that mature testing collaboration reduce late-stage defects and improve delivery confidence without sacrificing speed.

The Integrator Role

The QA Integration Lead

Connecting silos to create a unified delivery system.

Development

- Align test scope with sprints
- Pair on edge cases early

Business / Product

- Clarify acceptance criteria
- Risk-based prioritization
- UAT coordination

QA Integration Lead

Operations

- Orchestrate env readiness
- Validate test data needs

Release Mgmt

- Go/No-Go decision support
- Impact analysis of defects
- Release confidence scoring

Key Takeaway: Quality is not a role. It is a system.

Severity Levels for Faster, Clearer Triage

5 Severity Levels

From: **"Severity debates"** → To: **"Shared triage logic"**

Focus on impact and urgency

Showstopper

Testing cannot continue

Complete blocker; no workaround

Fix immediately

Ex: login down or QA build corrupted

Critical

Major business impact

Core flow broken; high customer risk

Fix within sprint

Ex: tax calculation wrong for all sales

High

Significant functional error

Testing continues, but release at risk

Fix before release

Ex: wrong price shown on sales page

Medium

Partial impact

Feature works with limitations

Schedule in sprint or next

Ex: slow dropdown or weak validation

Low

Minimal impact

Cosmetic or minor usability issue

Can move to backlog

Ex: tooltip typo or spacing issue

WHY THIS FRAMEWORK MATTERS

Predictable delivery is built on coordination, not control.

- Fewer blockers
- Cleaner prioritization

Decision Tree Impact

Before vs. After Using a Severity Decision Tree

● Triage Transformation

A shared decision tree turns **debate and delay** into **clarity and predictable delivery**

● Same logic across Dev, QA, Ops, Product

Before Using the Decision Tree	After Using the Decision Tree
– Severity debates in every triage meeting	✓ Instant, consistent severity classification
– Misaligned priorities across Dev, QA, and Ops	✓ Shared prioritization across all teams
– Critical defects mislabeled or overlooked	✓ Critical issues surfaced immediately
– Showstoppers discovered too late	✓ Showstoppers identified and escalated early
– Slow triage process with too much discussion	✓ Faster triage meetings with more action
– Unpredictable sprint outcomes	✓ Predictable, risk-based delivery

TRIAGE RESULT

Teams spend less time arguing about severity and more time resolving the critical path.

● Faster triage ● Higher release confidence

Why Dashboard Strengthens the QA Integration Lead Role

Data-Driven Quality

Leadership/Product
Strategy

- Leadership communication with clear metrics
- Risk-based prioritization Planning
- Executive-ready reporting
- Data-driven decision support
- Transparent accountability

QA Integration LEAD
Dashboard as the Operating System

Development
Engineering

- Live visibility into blockers Real-time
- Clear ownership paths for defects
- Go/no-go support with data-driven insights
- Cross-team dependency management
- Explicit escalation paths

Operations/Delivery
Infrastructure

- Environment readiness tracking Early
- Triage coordination across teams
- Release confidence scoring
- Operational constraints visibility
- Schedule alignment for smooth handoffs

Key Takeaway: **Dashboard becomes the operating system for collaboration, turning data into actionable quality insights.**

Section 2.3

Embedding Testing Across the Delivery Lifecycle

→ Put quality work where it **prevents risk**.

Lifecycle Integration

Where Testing Must Be Embedded

1

Requirements & Planning

- Testability reviewed alongside feasibility
- Acceptance criteria defined defined collaboratively
- Operational constraints identified early

2

Development Sprints

- Testing aligned to sprint cadence—not postponed
- Developers & testers pair on edge cases
- Early defect discovery reduces rework loops

3

Env & Ops Readiness

- Environment availability as a shared commitment
- Test data readiness tracked as dependency
- Ops engaged before crisis moments occur

4

Release & Go-Live

- Risk-based testing informs release decisions
- Clear escalation paths for critical defects
- Shared accountability for outcomes, not blame

Key Principle: Early alignment eliminates late surprises.

Cross-Team Collaboration

Tailored Cross-Team Meetings for Requirement Clarity

4 Meeting Types

From: **"Misaligned requirements"** → To: **"Shared understanding"**

Early alignment eliminates late surprises

Requirement Deep-Dive

Dev + QA + Product walk through stories together

Validate testability and acceptance criteria

Clarify user stories, edge cases, and integration points

Reduce requirement misunderstandings

Test Case Co-Creation

Developers & testers pair to define scenarios

Align on expected behaviors, negative paths

Identify edge cases and technical constraints

Faster test case creation with higher accuracy

Test Data Planning Huddle

Data team joins QA to map out data sets

Define data needs, mocks, and dependencies

Reduce last-minute scrambling for test data

Ensure data availability for testing

Environment Readiness Sync

QA + Ops confirm environment availability

Validate integrations and constraints early

Prevent environment blockers

Ensure predictable testing windows

Key Principle

Early alignment eliminates late surprises and reduces rework cycles.

- Fewer defects
- Faster sprints

Sharing Test Cases Early

When developers understand exactly how QA will validate their work, they write cleaner, more predictable code.

Early test case sharing transforms testing from reactive to preventive.

Developers Code With the End in Mind

Understanding validation expectations before coding reduces logic gaps and "works on my machine" surprises.

- Fewer logic errors
- Cleaner code

"Code with validation in mind"

Stronger Unit-Test Alignment

Unit tests align with QA validation scenarios, ensuring consistency between Dev and QA approaches.

- Better coverage
- Earlier defect detection

"Tests that match real scenarios"

Faster, Cleaner QA Readiness

Early test case sharing ensures test data and environments are ready before code arrives.

- Reduced rework
- Faster delivery

"Testing starts on time"

Key shift: Move from **"finding bugs"** to **"preventing risks"** — early test case sharing reduces defects and strengthens Dev-QA collaboration.

The most successful teams aren't the ones with the best tools.
They are the ones that collaborate early to prevent defects.

Before

"Can we test this?"

- Reactive firefighting
- Ad hoc environment fixes
- Late, defensive communication

→

After

"Are we ready to release?"

- Proactive readiness checks
- Daily cross-functional touchpoints
- Shared ownership of outcomes

Section 2.4

Coordinating Under Pressure: What Actually Works

→ From firefighting to **orchestration**.

Orchestration Framework

The Four Pillars of Coordination

01 Shared Visibility

- Unified defect dashboards across all teams
- Common status language (no ambiguous "done")
- Transparent risk indicators visible to leadership

02 Aligned Schedules

- Testing timelines synchronized with sprints
- Operational readiness checkpoints in planning
- No "surprise" testing windows or deployments

03 Proactive Communication

- Daily cross-functional touchpoints in critical phases
- Early escalation of blockers (hours, not days)
- Direct coordination—remove intermediaries

04 Clear Ownership

- Who owns environment readiness & data?
- Who prioritizes fixes vs features?
- Who accepts residual risk for release?

Predictable delivery is built on **coordination**, not control.

Analytics as the Backbone of Cross-Team Visibility

A practical system for unified QA, Dev, and leadership insight

Single source of truth. Faster triage. Predictable releases.

WHY DASHBOARDS MATTER

Why Unified Dashboards Matter

Transform scattered data into actionable insights

Visibility Framework

Single Source of Truth

Data Consolidation

Active

Consolidate Dev, QA, and Ops data into one real-time view

Eliminate manual reporting and inconsistent status updates

Remove scattered tools and duplicate data sources

Early Risk Signals

Proactive Detection

Alert

Surface blockers early before they become delays

Flag environment issues and integration failures

Consistent Definitions

Standardization

Aligned

Standardize severity definitions across teams

Align acceptance criteria with testability

Establish common status language

Executive-Ready Decisions

Leadership View

Ready

Provide real-time visibility for leadership

Enable data-driven go/no-go decisions

Deliver consistent executive reporting

Dashboard

What Dashboard Enables Across Dev & QA

Real-time Visibility

From: **"Can we test this?"** → To: **"Are we ready to release?"**

Dashboard turns signals into actions

Real-Time Defect Tracking

Auto-sync with seamless integration

Severity, priority, owner, and aging metrics

Highlight defects blocking test cases

Sprint alignment and traceability

Test Execution Heatmaps

Pass/fail trends by feature & environment

Identify unstable modules instantly

Visualize coverage gaps before release

Environment-based test analysis

Blocker Identification

Flags environment issues & data readiness

Integration failures highlighted

Delayed developer responses tracked

Missing test data identified

Predictive Insights

Defect aging predicts sprint spillover

Test execution velocity forecasts readiness

Environment uptime correlates with risk

Release confidence scoring

Key Insight

Dashboard turns signals into actions - transforming data into coordinated response.

- Real-time visibility
- Automated triage

Dashboard Layout | Unified Dev-QA View

Example Unified Dashboard Layout

Real-time Visibility

Defects by Severity
Critical: 3

Critical	High
3 ↓ 12%	8 ↑ 5%

Defect Aging Bar Chart

Aging Trend | Severity Map

Test Execution & Blocked
85% Pass

Executed	Blocked
85% ↑ 8%	12 ↓ 3

Execution Heatmap

Pass Rate | Blocked Tests

Env Uptime & Readiness
99.9%

Uptime	Readiness
99.9% ↑ 0.1%	92% ↑ 5%

Stability Timeline

Uptime Trend | Readiness Score

Release Confidence
95%

Confidence	SLA Breaches
95% ↑ 7%	2 ↓ 5

Risk Assessment

Confidence Score | SLA Status

Quality Health
Project Name - XYZ

> "Predictable delivery is built on coordination, not control."

Visual widgets: Defect aging bar chart, execution heatmap, blocker root-cause pie, response-time trend, environment stability timeline

Real-World Impact

What We Saw in Telecom Programs

Reduced Defect Turnaround Time

Rapid identification and resolution of critical blockers through direct cross-team coordination.

Prevented Last-Minute Rollbacks

Early risk detection and go/no-go alignment prevented expensive production production failures.

Increased Trust & Collaboration

Shifted culture from finger-pointing to shared problem solving between Dev, between Dev, Ops, and QA.

Predictable Delivery Under Pressure

Maintained release momentum and met deadlines despite high-stakes constraints.

> *"When teams move from firefighting to orchestration, pressure stops being a reason for failure and becomes a constraint we manage together."*
>
> Program Leadership Insight

Section 2.5

Coordinating Testing Across Global Teams

→ Design for distance.

Global Challenges

Global Reality: Added Complexity

Time Zone Misalignment

Limited overlap windows restrict real-real-time collaboration and delay handoffs.

Contractual Boundaries

Rigid scope definitions reduce flexibility during high-pressure crises.

Delayed Feedback Loops

Issues discovered overnight wait hours hours for resolution, slowing velocity. velocity

Cultural Differences

Varying communication styles impact how risks and blockers are escalated.

Dependency Gaps

Misalignment between internal teams and third-party vendors creates invisible risks.

Delivery speed is constrained by **handoff quality**, not geography.

Capitalizing on the Golden Hours

Tactical Execution

- Define overlapping golden hours strictly for live, cross-team collaboration and critical defect triage.
- Schedule testing cycles to mathematical maximize cross-team availability.
- Strictly avoid critical handoffs at the end of the reginal workdays to prevent 24-hours delays.

Delivery speed is **constrained**, by handoff quality, not geography..

Execution Framework

Five Strategies for Global Coordination

01

Time-Zone–Aware Planning

Define overlapping "golden hours" for live collaboration. Schedule testing cycles to maximize cross-team availability.

02

Clear Ownership

Establish a single point of accountability per function. Explicitly document ownership for environment readiness and defect triage.

03

Standardized Reporting

Use common defect severity definitions and maintain shared dashboards. Replace ad hoc updates with structured reporting.

04

Proactive Vendor Coordination

Engage vendors early in sprint planning. Share testing schedules in advance and validate readiness before critical milestones.

05

Cultural Awareness & Trust

Adapt communication styles to regional norms. Encourage psychological safety in escalation to reinforce shared goals.

Multinational Impact

Global Outcomes in Telecom

Fewer Overnight Delays

Drastically reduced downtime caused by unresolved defects sitting in queues across time zones.

Faster Vendor Responsiveness

External partners engaged earlier in planning cycles, treating them as true delivery partners.

Smoother Global Handoffs

Clear ownership and documentation ensured work continued seamlessly across regional boundaries.

Sustained Delivery Momentum

Maintained high velocity despite geographic complexity, distance barriers, and cultural differences.

> *"Global delivery succeeds when coordination is intentional, ownership is explicit, and communication is designed for distance."*

Key Takeaway

Documentation Strategy

Centralized Documentation with Content Management

Single Source of Truth

Before Content Management

Scattered & Outdated

Email Threads
Requirements buried in email chains, hard to find latest version

Personal Folders
Documents scattered across personal drives, no centralized access

Outdated Versions
Multiple versions of documents, confusion about which is current

Weak Traceability
No clear audit trail, hard to track changes and decisions

VS

After Content Management

Unified & Auditable

Single Repository
One location for requirements, test plans, data matrices, release notes

Version Control
Who/what/when/why tracked, sprint-aligned documentation history

Full Traceability
Requirements → Test Cases → Defects, complete audit trail

Audit Readiness
Transparent collaboration, versioned documents for compliance

Key Insight

Content Management becomes the centralized source of truth for delivery teams, eliminating outdated documentation and strengthening Dev-QA-Ops alignment.

End-to-End toolchain Software as the Testing Team's Visibility Engine

From: **"When is the code ready?"** → To: **"I can see exactly when it arrives"**

Real-time pipeline visibility

Story Readiness

Status: Ready

User story visibility - New → Active → Resolved → Ready for QA

Acceptance criteria linked to test cases

Testability validated before coding

Clear definition of "done"

Deployment Timing

Target: 2:00 PM

Build completion tracking - pipeline visibility

Deployment schedule to QA environment

Environment targeting (DEV → QA → STAGE)

Success/failure notifications

End-to-End Traceability

Coverage: 95%

Story → Test Case → Execution

Defect linkage to requirements

Full audit trail for compliance

Impact analysis for changes

Blocker Visibility

Escalation: Active

Missing test data alerts

Environment failure notifications

Early escalation for blockers

Cross-team dependency tracking

Key Insight

Aligned planning and predictable testing cadence eliminate late surprises

- Real-time visibility
- Faster triage

Key Takeaways

Summary: From Siloed to Synchronized

Building a collaborative testing culture across the lifecycle.

Why It Breaks

Silos create systemic failure. Misaligned
Misaligned priorities break delivery.

PROBLEM

MINDSET

Shared Ownership

Quality is a system, not a role. Move from
from gatekeeping to integration.

Embed Early

Eliminate late surprises.
Testability starts at requirements.

STRATEGY

EXECUTION

Coordinate Pressure

From firefighting to orchestration.
Visibility, alignment, communication.

Scale Globally

Design for distance. Intentional handoffs
and cultural awareness.

SCALE

Accelerator: Communication and coordination are the real drivers of velocity.

Next Step

Apply Monday

The Reality Check

Testing failures are rarely about tools, skills, or effort. They are almost always about **coordination**, **ownership**, and **timing**.

Shared Responsibility

Quality stops being a gatekeeper's burden and becomes a collective goal. **"Teams own outcomes."**

Proactive Communication

Shift from defensive explanations to early alignment. **"Align early, don't negotiate late."**

Predictable Releases

Delivery becomes a plan, not a stressful gamble. **"Built on coordination, not control."**

Thought

The most successful teams aren't the ones with the best tools. They are the ones that play as one.

Continuous Improvement

The Power of Sprint Lessons Learned

Transforming sprint pain into lifecycle improvement

Retrospective

Before

"Why does this keep happening?"

Recurring mistakes without systematic fixes

Recurring mistakes & repeated blockers
Same issues surface sprint after sprint

Blame culture & finger-pointing
"It's not my fault" mentality

Late escalations & crisis management
Issues discovered at end of sprint

After

"How do we prevent this?"

Pattern-based fixes & shared accountability

Pattern-based fixes & root cause analysis
Systematic problem solving

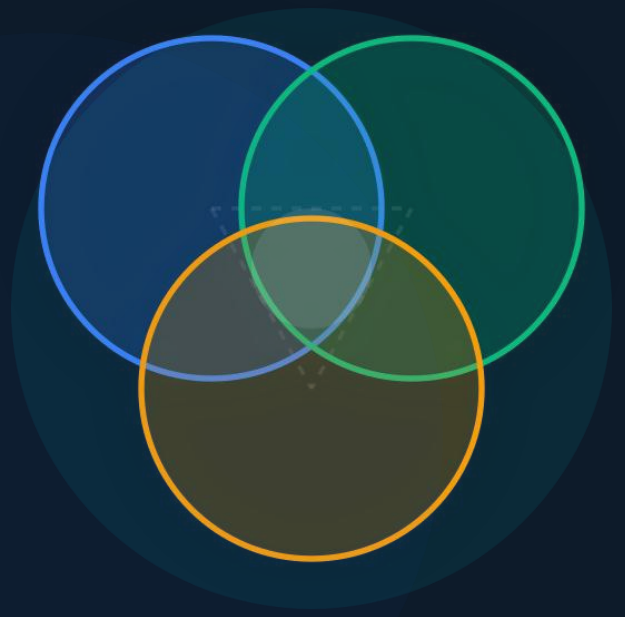
Shared accountability & ownership
Teams own outcomes, not blame

Faster future sprints & better readiness
Improved data & environment prep

Lessons learned sessions turn sprint pain into lifecycle improvement

Final Thought

"Quality is not owned by QA. Quality is owned by the delivery team."



Next Steps for Monday

Action 01

Pilot "Golden Hours"

Establish 2-3 hours of synchronous time for global teams.

Action 02

Unified Risk Dashboard

Create a single view for defects and environment status.

Action 03

Designate QA Integration Lead

Assign one person to bridge Dev, Ops, and Business.

Thank you for your time.

/in/qualityleader

hfontanes@ingelli.com

Thank you!

Date

April 29, 2026

StarEast Conference

Speaker

Haidee Fontanes

Process Improvement Manager/System Integration Lead